FORMULATRIX®

Software Engineer Bootcamp - Technical Document

MEETING ROOM BOOKING

Version	Date Presented	Author
Batch 18	26 May 2026	Denny Setyawan Huda
		Nathanael David
		Rifa Fauziah

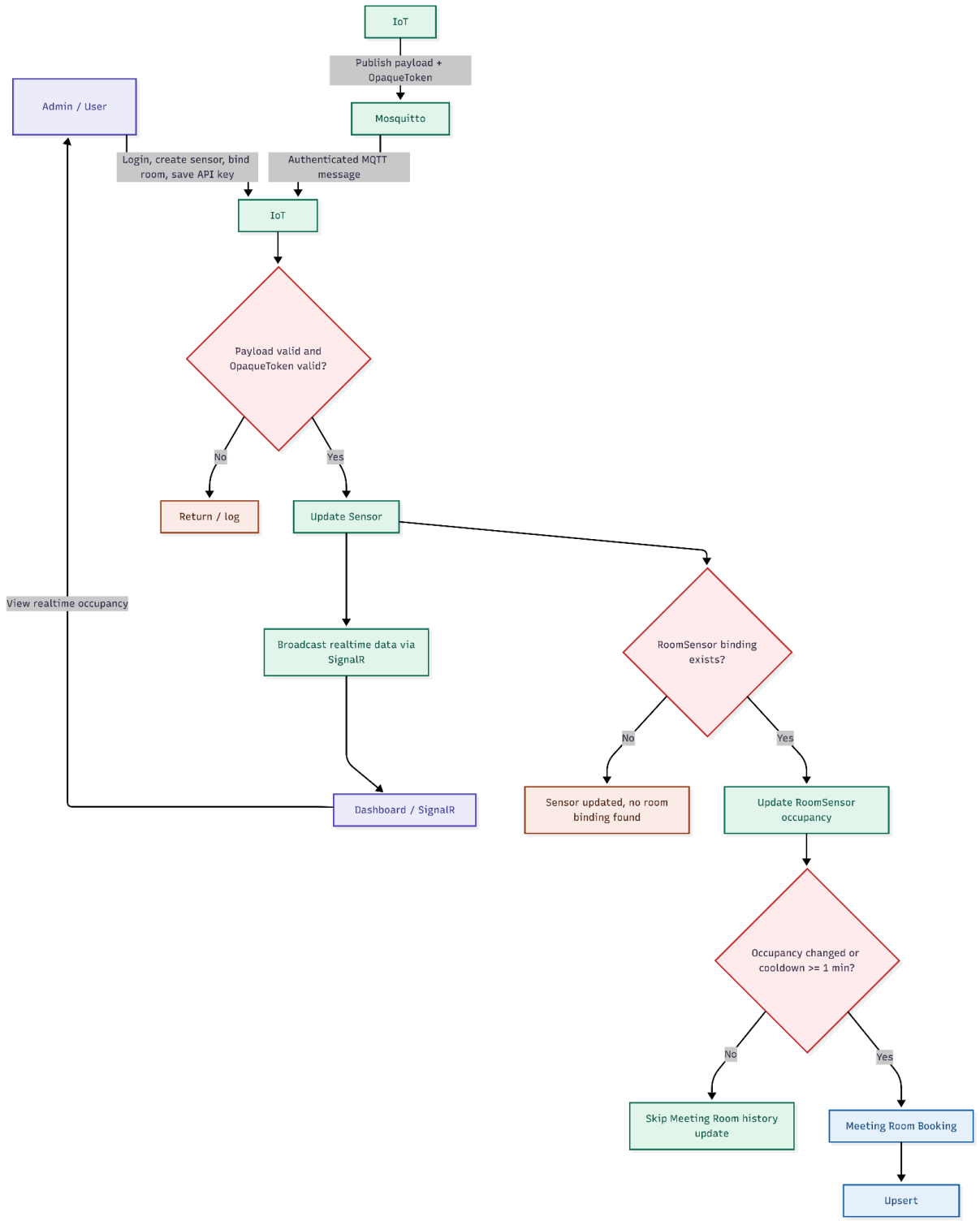
I. Introduction

Booking meeting rooms can be challenging when it involves a lot of people trying to book a limited number of meeting rooms. Problems arise such as schedule conflict, room maintenance, and making a facility report can impact productivity. To solve this problem, a web application for booking meeting rooms is made. With Google Calendar API integration, users can make reservations hassle-free. The existing Meeting Room Booking system helps users manage reservations, room information, and reports more efficiently.

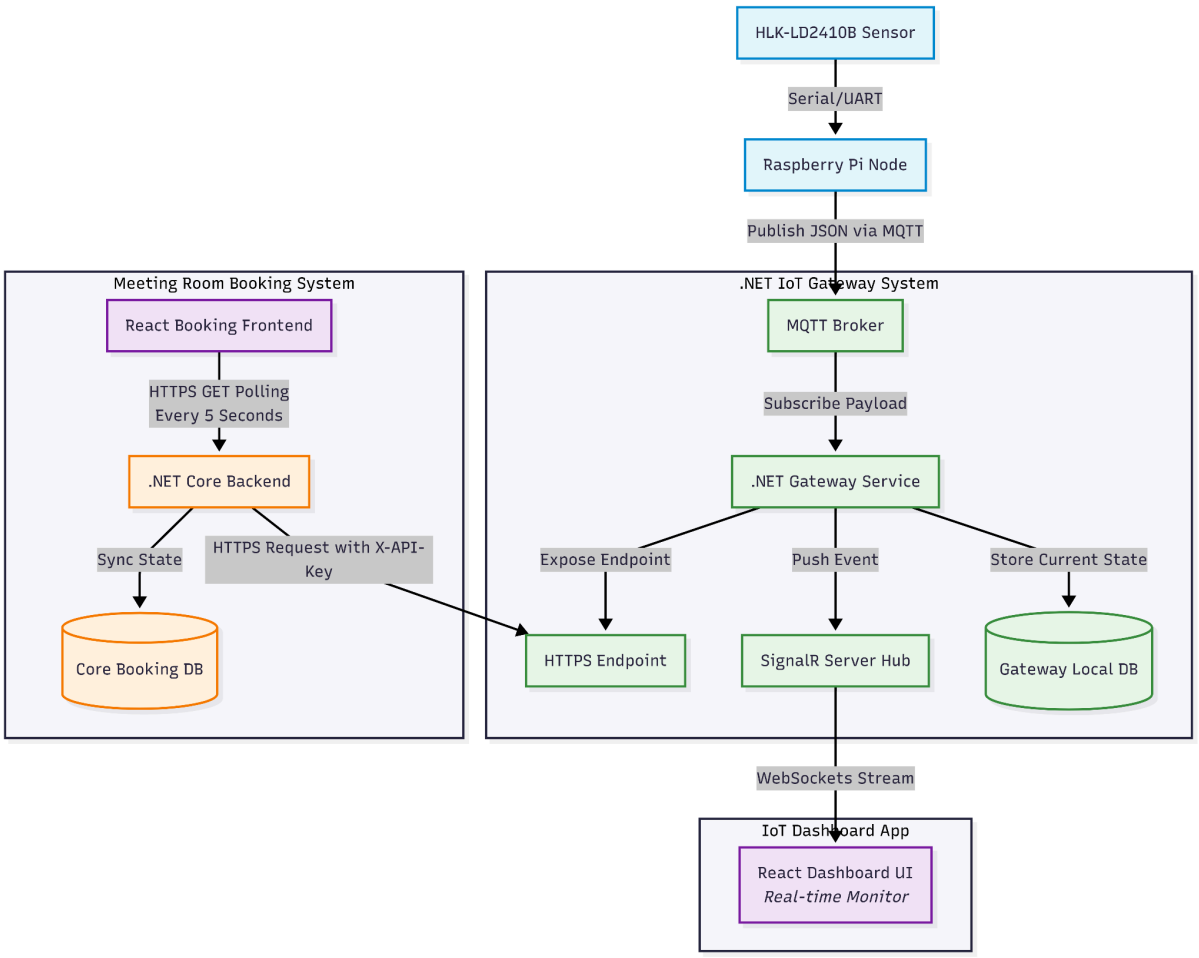
However, the existing system mainly relies on reservation data and does not fully represent the actual physical condition of each room. To improve this, IoT integration is introduced so the system can receive room condition data from sensors through an IoT Gateway. In addition, several bug fixes and flow enhancements are implemented to improve system stability, usability, and the overall booking experience.

II. IoT Integration System Architecture

User Flow Diagram



Data Flow Diagram



Explanation

The system architecture describes how the IoT components are connected to the existing Meeting Room Booking system. The architecture consists of several main components: sensor device, sensor application, MQTT broker, IoT Gateway backend, IoT Gateway web dashboard, Meeting Room Booking backend, Meeting Room Booking web application, SignalR, and database.

The sensor device detects room occupancy based on movement or presence inside the meeting room. The sensor application reads raw data from the sensor through a serial connection, parses the sensor frame, validates the data, and converts it into a structured payload. The payload includes occupancy status, moving target distance, stationary target distance, sensor connection status, and timestamp.

After the payload is created, the sensor application publishes the data to the MQTT broker. The IoT Gateway backend subscribes to the MQTT topic and receives incoming sensor payloads. The gateway authenticates the device using an opaque token, validates the payload, processes the occupancy status, and checks whether the sensor is already bound to a meeting room.

If the sensor has a valid room binding, the IoT Gateway maps the sensor data to the correct meeting room and sends the occupancy update to the Meeting Room Booking backend through HTTP API communication using API key authentication. The Meeting Room Booking backend receives the data and stores it as device history in the database.

SignalR is used inside the IoT Gateway system to deliver real-time sensor updates from the IoT Gateway backend to the IoT Gateway web dashboard. This allows administrators to monitor sensor changes without manually refreshing the dashboard. Meanwhile, the Meeting Room Booking web application remains the main application for users to manage reservations, reports, and other room-related activities.

Flow

1. Sensor devices detect movement or presence inside the meeting room.
2. Sensor application reads raw data from the sensor through a serial connection.
3. Sensor application parses and validates the raw sensor frame.
4. Sensor application creates a structured sensor payload.
5. Sensor application publishes the payload to the MQTT broker.
6. IoT Gateway backend subscribes to the MQTT topic and receives the payload.
7. IoT Gateway authenticates the device using an opaque token.
8. IoT Gateway processes the occupancy data and checks the room-sensor binding.
9. IoT Gateway sends real-time sensor updates to the IoT Gateway web dashboard through SignalR.
10. IoT Gateway sends valid occupancy updates to the Meeting Room Booking backend using HTTP API with API key authentication.
11. Meeting Room Booking backend stores the data as device history.
12. Meeting Room Booking web application remains the main interface for reservations, reports, and room-related activities.

III. Key Feature

1. Sensor System

Problem

The existing Meeting Room Booking system could not detect the actual physical condition of a meeting room. It only relied on reservation data, so the system could not determine whether a room was actually occupied or empty. Raw data from the sensor also could not be used directly because it needed to be read, parsed, validated, and converted into a structured format.

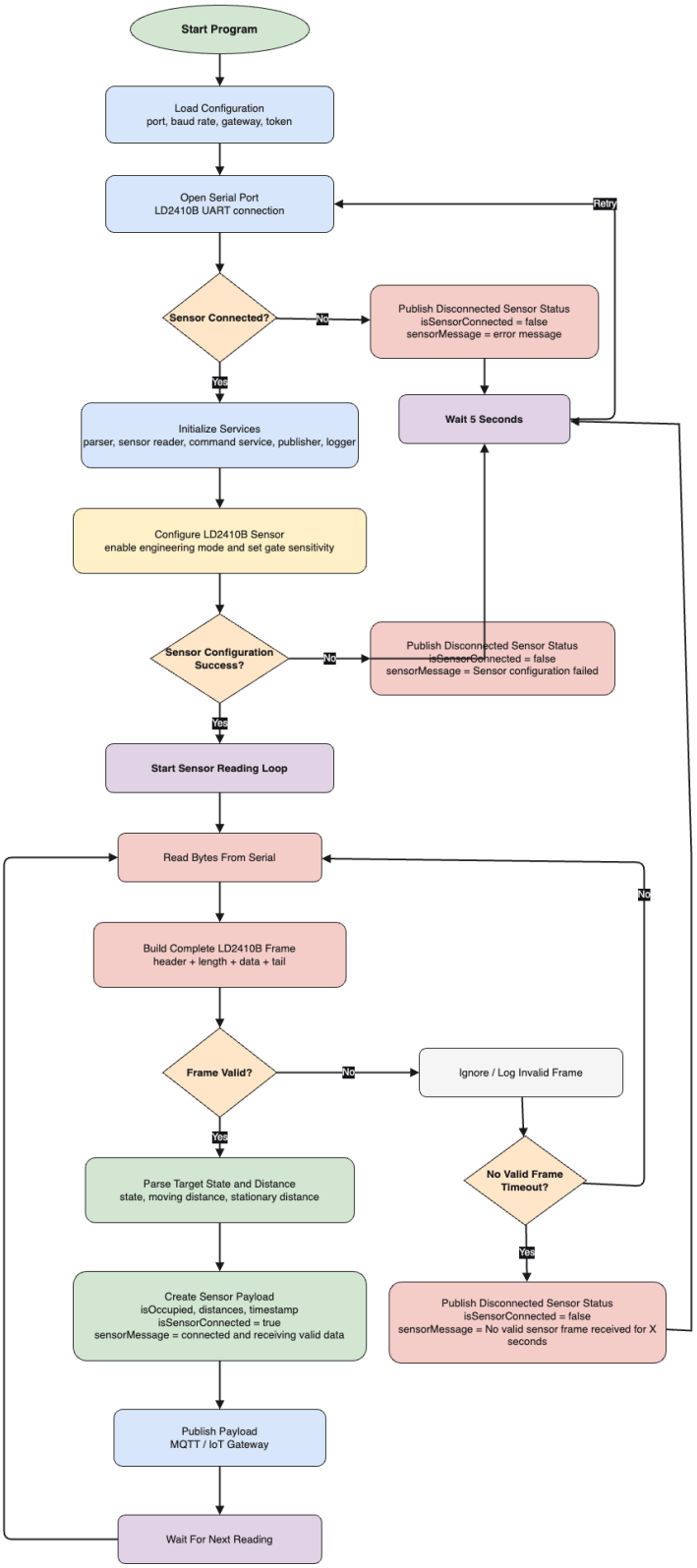
Solution

A sensor application was developed to read room occupancy data from the sensor device through a serial connection. The application continuously receives raw sensor frames, parses the incoming data, validates the sensor connection state, and determines the room occupancy status based on detected movement and stationary target values.

After processing the sensor data, the application creates a structured payload containing occupancy status, moving target distance, stationary target distance, sensor connection status, sensor message, and timestamp. This payload is then published to an MQTT broker, allowing the IoT Gateway to consume the data and forward it to the Meeting Room Booking system. This integration enables the system to use real-time sensor information as an additional source of truth for room occupancy monitoring.

Diagram

Data Program Flow Diagram



2. IoT Gateway

Problem

Sensor data cannot be used directly by the Meeting Room Booking system without a middleware layer. The system needs a component that can receive sensor data, authenticate the device, validate the payload, process occupancy information, and map the sensor to the correct meeting room.

Without an IoT Gateway, sensor data would be difficult to manage and integrate with the existing Meeting Room Booking system. Administrators would also have no centralized interface to manage sensor devices, configure gateway settings, bind sensors to rooms, or monitor real-time sensor updates.

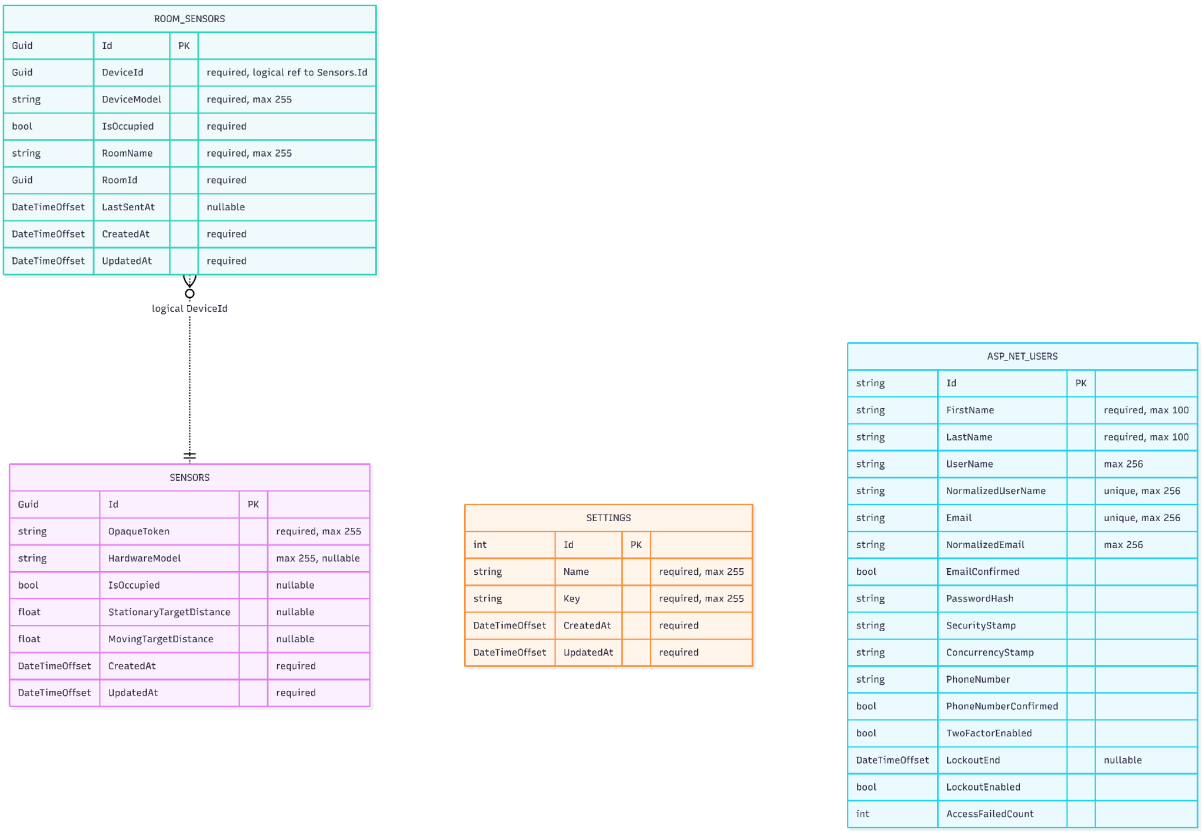
Solution

The IoT Gateway was implemented as the bridge between the sensor system and the Meeting Room Booking backend. It receives sensor data from the MQTT broker, authenticates the device using an opaque token, validates and processes the incoming payload, and checks whether the sensor is bound to a meeting room.

If the sensor has a valid room binding, the IoT Gateway maps the sensor data to the correct meeting room and sends the occupancy update to the Meeting Room Booking backend through HTTP API communication using API key authentication. The gateway also stores and manages sensor data, room-sensor bindings, and gateway configuration.

The IoT Gateway also provides a web dashboard for administrators. Through this dashboard, administrators can manage sensors, configure gateway settings, bind sensors to meeting rooms, and monitor sensor data. SignalR is used to deliver real-time sensor updates from the IoT Gateway backend to the IoT Gateway web dashboard. Docker and Docker Compose are used to simplify the setup of the gateway backend, gateway web dashboard, MQTT broker, and related services.

IotGateway Database Schema



3. Meeting Room Booking Integration

Problem

The main application framework of the Meeting Room Booking system cannot communicate directly with the MQTT broker or physical sensor nodes due to network isolation and architectural differences. To display active room usage, the React frontend requires a reliable, secure mechanism to frequently fetch the actual room occupancy status and historical device logs without causing database locks or high processing overhead on the core server.

Furthermore, integrating real-time physical statuses requires an automated device-to-room binding synchronization. Without a proper authentication layer, exposing the IoT Gateway endpoints to the core booking backend would introduce major security vulnerabilities, allowing unauthorized data manipulation or unauthorized access to the local gateway network.

Solution

The integration was achieved by establishing a secure communication bridge between the .NET Core Booking Backend, the React Frontend, and the .NET IoT Gateway using an optimized API architecture. An X-API-Key authentication system was implemented within the Meeting Room Booking platform; the generated token is securely masked and displayed only once to administrators for binding setup, ensuring authorized-only requests between the core backend and the IoT Gateway.

To keep the user interface updated without exhausting system resources, a decoupled data synchronization strategy was applied:

1. **The Core Backend Integration:** The .NET Core Backend securely communicates with the .NET IoT Gateway via HTTPS requests using the generated X-API-Key to pull structured room list mappings and the latest sensor frames.
2. **The Frontend Integration:** The React-based Booking Web Application implements an HTTPS short-polling mechanism that queries the .NET Core Backend every 5 seconds. This updates the specific room's detailed view, displaying the active IsOccupied status and the historical timeline of the DeviceHistory logs

4. Report Management System Enhancement

Problem

The existing report management flow was not fully clear for both users and administrators. Report priority values were displayed as Unknown because the frontend did not properly align its priority mapping with the enum values returned by the backend. Although report status already existed in the backend, administrators did not have a clear frontend control to update report progress. The reporting flow also had usability gaps. The report issue action on the Room Detail page was not visible enough, so users could easily miss it. In addition, the follow-up process was not fully accessible from the frontend. Administrators needed a clearer way to add progress comments, while report owners needed better visibility into how their submitted issues were being handled. The report resolution flow also lacked supporting context. Users could see that a report was resolved, but they could not clearly see what action had been taken. There was also no structured way for users to provide feedback after a report was resolved, and photo evidence was not fully integrated into the report creation and detail viewing flow.

Solution

The report management system was enhanced to improve visibility, communication, and issue tracking transparency. Report priority mapping was fixed so that Low, Medium, High, and Critical are displayed correctly across the report list and update modal. An admin-facing status update control

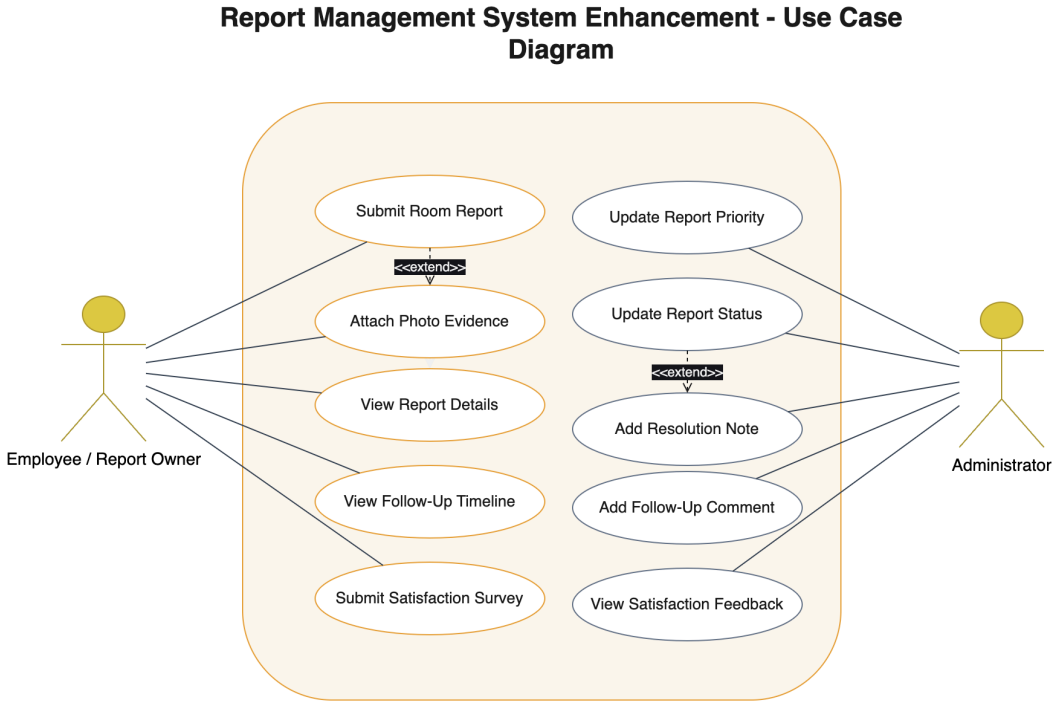
was added, allowing administrators to update report statuses such as Pending, In Progress, Resolved, and Closed directly from the report list, while regular users can only view the current status.

To improve report submission, a clearer report issue action was added to the Room Detail page so users can submit room-related reports directly from the room context. A report follow-up modal was also added to display a chronological timeline of report events, including submissions, status changes, detail updates, resolution notes, and admin comments. Administrators can add follow-up comments, and report owners receive in-app notifications when new comments are added.

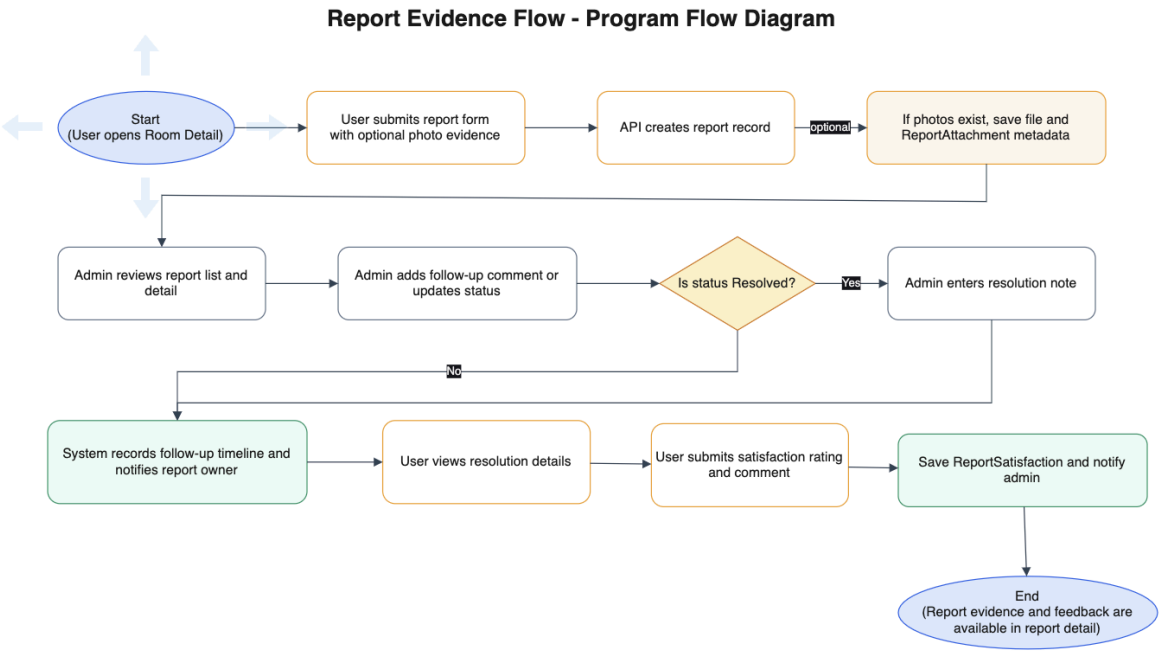
The resolution flow was improved by requiring administrators to provide a resolution note when resolving a report. This note is stored in the follow-up timeline so users can understand what action was taken. A satisfaction survey was also added, allowing report owners to submit a one-time rating and optional comment after their report is resolved. This feedback is stored in a dedicated ReportSatisfaction table and can be viewed by administrators.

Photo evidence support was added to allow users to attach optional images when creating reports. Uploaded photos are linked to the report and displayed in the report detail modal, keeping the main report table clean while still making supporting evidence accessible. Together, these improvements create a more complete, transparent, and user-friendly report management workflow.

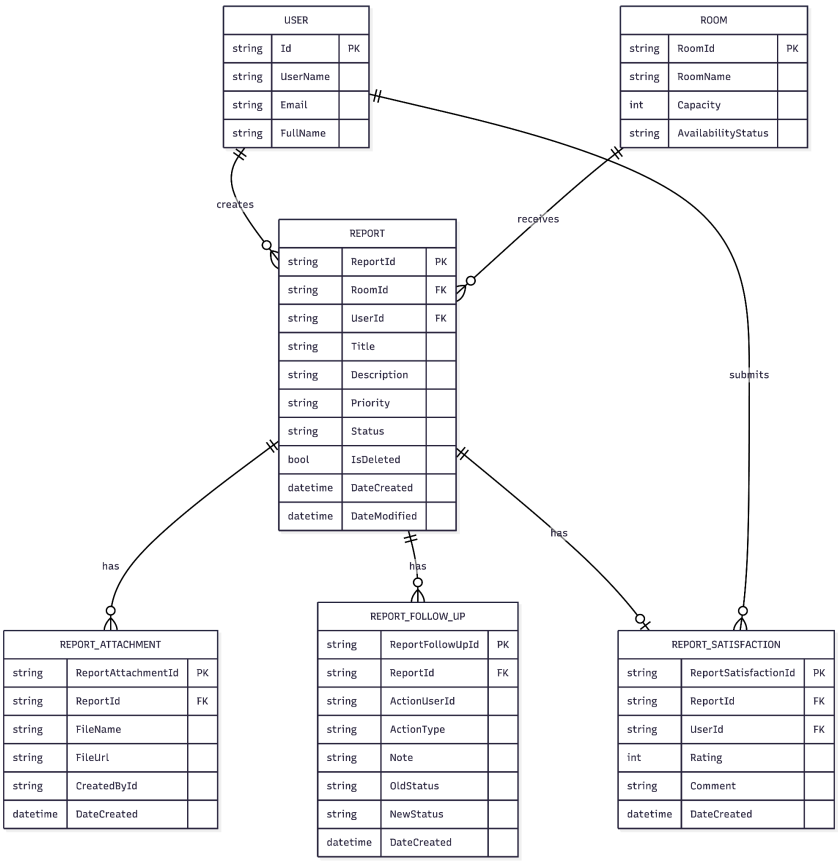
Use Case Diagram



Program Flow Diagram



ERD



Note: This feature only adds “Report_Satisfaction” and the other database already existed.

5. Testing

Problem

The addition of IoT integration, IoT Gateway services, report flow enhancements, and bug fixes increased the risk of regressions in the existing Meeting Room Booking system. Since the system had already been developed by previous batches, new changes needed to be validated carefully to ensure they did not break existing functionality. The IoT flow also introduced several integration points, including sensor frame parsing, MQTT publishing, gateway ingestion, device authentication, room-sensor binding, API communication, and device history storage.

Without proper testing, invalid sensor data, failed gateway processing, incorrect room binding, or broken API communication could cause inaccurate room occupancy information. In addition, changes to the report flow and existing bug fixes could affect user experience, data consistency, and system stability if not properly verified.

Solution

Testing was expanded across the sensor application, IoT Gateway, and Meeting Room Booking system. The sensor tests validate configuration, serial communication, frame parsing, sensor command behavior, MQTT publishing, and runtime service behavior. IoT Gateway tests cover authentication, sensor management, ingestion service, processing service, room-sensor binding, Meeting Room Booking integration, and repository logic.

Testing was also performed on the Meeting Room Booking backend to validate IoT-related endpoints, API key authentication, device history storage, report flow enhancement, and bug fix behavior. These tests help ensure that new IoT features and system improvements work correctly while maintaining the stability of the inherited Meeting Room Booking application.

IV. Individual Contribution

1. Denny Setyawan Huda

A. Iot Gateway - Database Schema & API Endpoint Development (Backend)

1. **[OP-146761] [API] Create Get Room Sensor Endpoint**
   Created a Merge Request introducing a new API endpoint to retrieve sensor data filtered by a specific room. The changes include implementing GetRoomSensorsAsync in ProcessingService and adding the GetRoomSensors endpoint in ProcessingController with standardized REST API response format.

2. **[OP-146939] [API] Add Room Name to Room Sensor Table**
   Submitted a database migration and backend update via a Merge Request to add the RoomName column directly into the RoomSensor entity. The changes include updating DTOs, refactoring CreateRoomSensor to use a request DTO, and standardizing API success responses across the service layer.

3. **[OP-147001] [API] Create Get Sensor by Id**
   Passed a Merge Request that implements a dedicated API endpoint to fetch detailed information for a single sensor using its unique ID. The changes include adding a new endpoint to retrieve sensor by ID and enriching the response DTO with detailed sensor properties for better client visibility.

B. Iot Gateway - Core Service Refactoring & Payload Optimization (Backend)

1. **[OP-145385] [API] Fix Sensor Request DTO (Adding HardwareModel Field)**
   Submitted a Merge Request to update the Sensor Request DTO schema by adding the HardwareModel field and enhancing DeviceCreateDto. The changes also include cleaning up unused User controller, service, repository, and DTO files to streamline the codebase.

2. **[OP-146585] [API] Refactor Setting Service**
   Processed a comprehensive Merge Request to refactor the Setting Service codebase. The changes include renaming GetSettingAsync to GetFirstSettingAsync, consolidating setting repository retrieval methods, and updating SettingController to remove unused endpoints.

C. Iot Gateway - Gateway Configuration & Room Setup Views (Frontend)

1. **[OP-145212] [SETUP] React Repository Architecture**
   Submitted a foundational Merge Request to set up the React repository architecture with Vite and TypeScript. This MR introduces a structured folder layout with shadcn UI components, authentication context, routing with protected routes, and base pages for dashboard, devices, and home.

2. **[OP-145432] [WEB] Implement API Gateway Service**

Created a Merge Request to implement the API Gateway service on the client side. The changes introduce a centralized service layer for auth and sensor APIs, add new UI components (dialog, select, skeleton, sonner), enhance the sidebar navigation, and update device pages with improved CRUD operations.

3. **[OP-146589] [Web] Create Setting Configuration Page**
   Submitted a Merge Request to develop the user interface for the Setting Configuration page. The changes include building a responsive form with API credential fields, implementing useCreateSetting hook with React Query, and adding the setting service for backend communication.

4. **[OP-146768] [WEB] Get Room Sensor Page**
   Processed a Merge Request to build the Room Sensor page interface with full CRUD capabilities. The changes include implementing RoomSensorsPage with data lists, CreateRoomSensorPage with form handling, custom hooks (useGetRoomSensors, useCreateRoomSensor, useDeleteRoomSensor), and a dedicated room sensor API service.

5. **[OP-146920] [WEB] Implement Bind Room Sensor Page**
   Passed a Merge Request implementing the interface and logic for the Bind Room Sensor page. The changes replace the manual Room ID input with a dynamic dropdown that fetches available meeting rooms, display current gateway settings, and integrate React Query hooks for meeting room and setting data.

D. Iot Gateway - Real-Time Stream & Device Monitoring (Frontend)

1. **[OP-147009] [WEB] Implement Real Time Device with SignalR**
   Submitted a major Merge Request to implement a real-time device monitoring page with SignalR. The changes include creating a dedicated single-device page with live sensor telemetry, adding sensor detail fetching hook, integrating chart.js for data visualization, and removing the old SignalR test page.

2. **[OP-147024] [WEB] Fix Real Time with SignalR**
   Created a follow-up Merge Request to refactor the device page for exclusive real-time data streaming. The changes include optimizing the single-device page layout, renaming useGetSensor to useGetSensorId, cleaning up MeetingRoomService and OpaqueTokenService, and removing the IotGateway.Tests project from the solution.

E. Meeting Rooom Booking - Room Device Logging & Data Structure (Backend)

1. **[OP-145206] [SETUP] Create DeviceHistory Table**
   Submitted a foundational Merge Request to set up the database schema by creating the DeviceHistory table along with its entity definition, DbSet configuration, and relationship mappings in the DbContext. This MR establishes

the primary data structure required to log, track, and store chronological history logs for IoT devices in meeting rooms, including fields for device identifiers, sensor readings, timestamps, and room associations that form the backbone of the entire device logging system.

2. **[OP-145509] [API] Implement Post Device History**
   Merged a request that introduces a new POST API endpoint designed to receive and process device logs from IoT gateways. This implementation encompasses the entire stack, from the controller action and service layer down to the repository method. It enables the validation, processing, and successful storage of incoming data packets that contain room sensor readings into the DeviceHistory table in the database.

F. Meeting Room Booking - API Security & Code Optimization (Backend)

1. **[OP-145310] [API] Implement IoT API Key to Protect Endpoint**
   Created a Merge Request to implement API key authentication middleware (IotApiKeyAttribute) that secures backend communication between IoT gateways and the API. This MR introduces the full authentication flow—including the ApiKeyGenerator for creating secure keys, HashApiKey for one-way hashing, the IotApiKey entity and repository for persistent storage, and the custom attribute applied to controllers—ensuring only verified IoT gateway clients with a valid key can read or write device data.

2. **[OP-145556] [API] Remove Unnecessary Try Catch**
   Submitted a code cleanup Merge Request to remove redundant try-catch blocks scattered across the IotController and its underlying service and repository layers. This optimization reduces code duplication, improves overall readability and maintainability, and relies on the global exception handling middleware already in place to catch and process any unhandled errors consistently.

3. **[OP-146074] [API] Change Sort by UpdatedAt**
   Processed a Merge Request to modify the API query sorting logic in the DeviceHistory data retrieval, changing the order criteria from CreatedAt to the UpdatedAt field. This ensures that any data requests fetching device history logs return the most recently modified records first, giving users and administrators a more accurate and timely view of room sensor activity.

G. Meeting Room Booking - IoT Gateway Integration & Device Status UI (Frontend)

1. **[OP-146008] [WEB] Integration Status Room from IoT Gateway**
   Submitted a merge request to implement the client-side integration that will fetch and display real-time room occupancy statuses from the IoT gateway. This integration effectively connects the frontend Room model and RoomDetail component with live sensor data, introducing a clear visual status indicator such

as "occupied" or "vacant." This enhancement empowers users to instantly determine if a meeting room is available, eliminating the need to consult external systems.

2. **[OP-146071] [WEB] Add Health Check IoT Gateway API**
   Submitted a merge request that introduces a frontend health check indicator to the admin dashboard. This new monitoring feature regularly performs connectivity checks against the IoT Gateway API service endpoint and displays the current network status, indicating whether it is online, offline, or unreachable. This enhancement enables administrators to quickly identify and address any connectivity issues with the gateway infrastructure.

3. **[OP-147154] [WEB] Implement Device History in Detail Room**
   Submitted a Merge Request to build a dedicated data log table component embedded inside the Room Detail page view. This UI enhancement fetches historical records from the DeviceHistory table via the existing GET endpoint and renders them in a paginated, sortable table format, giving users and administrators full visibility into the chronological sensor activity for any selected meeting room.

H. Meeting Room Booking - Security Management & UI Refinement (Frontend)

1. **[OP-146021] [WEB] Api Key Generate Page**
   Created a Merge Request to design and develop the complete interface for the API Key Generation page. This component includes a dedicated form where administrators can trigger secure token creation via the backend endpoint, a display area showing the generated key, and a one-click copy button that safely copies the access key to the clipboard, along with proper navigation integration in the admin menu.

2. **[OP-146209] [BUG] Fix Group Page**
   Submitted a bug-fix merge request to resolve layout constraints and data rendering issues on the Group Management page. The fix addresses incorrect component alignment, missing property mappings in the group member interface, and improper state handling that caused the group list to display incorrectly. As a result, this update restores smooth UI navigation and ensures reliable rendering of the group list data.

3. **[OP-147110] [BUG] Implement Logout Functionality Without Page Refresh**
   Successfully processed a merge request to enhance the logout system flow, ensuring that signing out no longer triggers a full browser page reload. The updates effectively clear locally stored authentication tokens, reset the global application state through the authentication context, and seamlessly redirect users to the login page. This entire process is handled asynchronously within the

existing modal flow, delivering a significantly smoother and more responsive user experience.

2. Nathanael David

A. Foundation, Architecture, and Refactoring

1. **[OP-144850][Setup] Solution Structure for all Service**

   Created the initial IoT Gateway solution structure and base backend project organization. This included early controller, DbContext, interface, DTO, and service files used as the first foundation of the API.

   The work made later feature development easier because the project already had a separated structure for controllers, services, repositories, DTOs, and shared service responses.

2. **[OP-145392][API] Refactor all namespaces in IotGateway.Api**

   Standardized namespace usage and folder organization across the API project. This included cleaning up DTOs, mappers, interfaces, controllers, services, and incorrect folder naming such as “Mapppers” to Mappers.

   The refactor improved maintainability before the codebase became larger with authentication, room binding, MQTT, settings, SignalR, and Docker changes.

3. **[OP-146758][API] Refactor IotGateway.Api codebase for clean code**

   Refactored the API codebase to improve readability and consistency. The related branch history includes global using cleanup, DTO constructor adjustments, CORS updates, and cleaner header/token handling.

   This reduced repeated using declarations and made the API easier to extend. It also helped standardize patterns before adding and maintaining later features.

4. **[OP-145353][API] Creating ExceptionHandler for global Exceptions**

   Created a global exception handling middleware for the API and wired it into the middleware pipeline.

   This centralized unexpected error handling so the API can return more consistent responses and reduce repeated try/catch-style behavior across controllers and services.

B. Database and Persistence

1. **[OP-145020][Database] Create Database schema for device registry**

   This work introduced the early Sensor model, DbContext setup, repository/service structure, mapper setup, and validation around sensor data. It became the foundation for storing registered devices and later updating their state from ingestion payloads.

2. **[OP-146079][Database] Create ApiKey Table for API Storage**

   Added the first database support for storing API keys used by the Meeting Room Booking integration. The implementation introduced an ApiKeys table and related model, service, repository, DTO, mapper, and validation support.

   This enabled the IoT Gateway to persist external service credentials instead of relying only on static configuration. In the latest code, this earlier ApiKey concept has been replaced by Setting.

3. **[OP-146480][Database] Rename IotKey/ApiKey Model to Setting**

   Renamed the API key/configuration model into Setting. The related commits update migrations, DbContext, DTOs, repositories, services, mappers, global usings, and Meeting Room integration references.

   The rename made the model responsibility clearer because the stored value represents gateway configuration for Meeting Room Booking access, not just a generic API key table.

C. Authentication and Device Security

1. **[OP-145215][API] Implement User Authentication**

   Implemented user authentication using ASP.NET Identity and JWT. This included login/register DTOs, validators, JWT generation, auth mapper, user repository, auth service, and authentication registration in Program.cs.

   This made it possible to protect administrative endpoints such as sensor management, setting management, room binding, and user profile retrieval with JWT authentication.

2. **[OP-145217][API] Implement Opaque Token Authentication**

   Added the first database support for storing API keys used by the Meeting Room Booking integration. The implementation introduced an ApiKeys table and related model, service, repository, DTO, mapper, and validation support.

   This enabled the IoT Gateway to persist external service credentials instead of relying only on static configuration. In the latest code, this earlier ApiKey concept has been replaced by Setting.

3. **[OP-145387][API] Delete Register Endpoint and Implementing Seeder for User**

   Removed public registration behavior and replaced user initialization with database seeding. The related branch/history appears as OP-145385_refactor-register-add-seeder, while OpenProject shows OP-145387.

This improved security because new users are no longer created from an open registration endpoint. The application instead starts with seeded admin data for predictable access during development and deployment.

4. **[OP-146001][Security] Increase OpaqueToken Security for Better Authentication**

   Improved opaque token authentication security by strengthening token service logic and removing older token helper behavior.

   The change reduced inconsistent token usage and made device authentication safer. It also made the token validation flow easier to maintain.

5. **[OP-146010][API] Fix OpaqueToken usage in Controller and Service layer**

   Fixed opaque token usage across controller and service layers. The related branch/history appears as OP-146014_fix-opaquetoken-usage-controller-service, while OpenProject shows OP-146010.

   This aligns how raw and hashed opaque tokens are handled in authentication, credential lookup, ingestion, and service calls. The result is a more consistent device authentication flow.

6. **[OP-145351][API] Implement GET /api/me**

   Implemented authenticated user profile retrieval. The related branch/history added user service/repository support, user DTOs, mapper/validator setup, and secured user controller behavior.

   This lets the web dashboard retrieve user data for the active session. In the latest code, this flow is exposed through /api/user/me, where the backend reads the current user ID from JWT claims.

D. Sensor Device Management

1. **[OP-145213][API] Implement POST /api/sensor**

   Implemented the sensor registration endpoint through POST /api/sensor. This work added device creation DTOs, sensor service logic, repository persistence, mapper updates, and token generation/hashing behavior.

   When a sensor is registered, the API returns the raw opaque token once and stores only the hashed token in the database. This endpoint is the main entry point before a device can publish valid telemetry.

2. **[OP-145302][API] Implement GET /api/sensor**

   Implemented retrieval of registered sensors through GET /api/sensor. The related work added service and controller methods for listing sensor/device records.

   This endpoint supports the web dashboard device list. It allows users to see registered devices and their latest known telemetry fields.

3. **[OP-145304][API] Implement DELETE /api/sensor/{id}**

   Implemented sensor deletion by ID. The controller and service were updated so administrators can remove registered devices from the gateway.

   This completed the basic sensor lifecycle with create, read, and delete operations. It also helped the frontend provide device management actions.

4. **[OP-145443][API] Strip HashedToken From SensorResponseDto.cs**

   Removed sensitive token information from normal sensor responses. The related changes introduced safer separation between creation/auth DTOs and normal device response DTOs.

   This prevents hashed or raw credential data from leaking into device list/detail responses. Token material is only exposed when required, such as during device creation.

5. **[OP-145499][WEB] Refactor SensorResponse at client side**

   Updated frontend sensor response handling to match backend DTO changes. Device cards and client-side services were adjusted so they no longer depend on opaque token fields in normal sensor responses.

   This kept the UI aligned with the backend security improvement and improved how device IDs and sensor details are displayed on the dashboard.

E. Ingestion and Occupancy Processing

1. **[OP-145214][API] Implement POST /api/ingestion**

   Implemented sensor data ingestion through POST /api/ingestion. The endpoint receives sensor telemetry, validates the payload, resolves the authenticated device, and updates sensor state.

   This endpoint is used for HTTP-based ingestion. MQTT ingestion later reuses the ingestion service after messages are received from Mosquitto.

2. **[OP-145622][API] Implementing POST /api/ingestion with sending RoomSensor data to MeetingRoom**

   Enhanced ingestion so sensor updates can also update room-sensor data. The implementation connected sensor telemetry with room binding information and prepared room sensor response data.

   This made occupancy processing more useful because sensor updates can affect a room occupancy state and can be forwarded to Meeting Room Booking when needed.

3. **[OP-145718][API] Fix IngestionService.cs for request to MeetingRoomBooking**

   Fixed the ingestion-to-Meeting Room Booking request flow. The related commits improve unchanged occupancy handling and correct device model mapping before external requests are sent.

This prevents unnecessary external calls while keeping occupancy history accurate. It also improves reliability when syncing room sensor history.

4. **[OP-145814][API] Fix Validation at IngestionService Request**

   Improved validation and state-change handling inside IngestionService. The changes make incoming sensor payloads safer before database updates or external API calls happen.

   This reduces invalid data processing and helps ensure Meeting Room Booking receives valid occupancy history data.

5. **[OP-145999][API] Fix IngestionService Flow and RoomSensor field to improve data consistency**

   Added and wired the LastSentAt field for RoomSensor. The related commits update model, DTO, migration, and repository/service update behavior.

   LastSentAt is used to throttle Meeting Room Booking history updates. The gateway can update local state frequently while avoiding repeated external history writes within the cooldown window.

F. Room Binding and Meeting Room Booking Integration

1. **[OP-145438][API] Create endpoint for getting Room from MeetingRoomBooking**

   Implemented the Meeting Room Booking integration for retrieving room data through the IoT Gateway API. The related work added MeetingRoomController, MeetingRoomService, Meeting Room API base URL configuration, validation, and response DTO handling for room data.

   The original OpenProject title mentions getting an API key, but the latest implementation no longer exposes an API key endpoint for this flow. Meeting Room credentials are configured through POST /api/setting, stored in the Settings table, and read by MeetingRoomService via ISettingRepository when calling api/iot/rooms.

2. **[OP-145442][API] Strip Meeting Room calendarId from RoomResponseDto**

   Removed unnecessary CalendarId information from RoomResponseDto. The gateway only needs fields relevant to room identification and room-sensor binding.

   This simplified the API response shape and avoided exposing data that the IoT Gateway does not use directly.

3. **[OP-145507][API] Create endpoint for Binding Sensor with Room**

   Implemented the backend room-sensor binding flow. The work added RoomSensor model/table, repository, mapper, request/response DTOs, validation, processing service, and bind/unbind endpoints.

This feature connects a physical sensor to a meeting room. Once bound, sensor telemetry can update room occupancy and create history for Meeting Room Booking.

4. **[OP-145936][API] Implement GET Device History**

   Implemented an authorized GET /iot/device-history API endpoint for retrieving device history by room. The work introduced GetDeviceHistoryRequest for RoomId, Page, and PageSize query parameters, and DeviceHistoryResponse for returned device history data.

   The implementation adds service and repository methods to validate the room, fetch matching device history records, sort them from newest to oldest, and apply pagination. It also adds AutoMapper support and includes a migration that changes timestamp fields from DateTimeOffset to DateTime.

5. **[OP-147054][WEB] Add Select Sensor using GET Sensor at Binding Room Sensor**

   Improved the room binding UI by adding dynamic sensor and room selection. The related branch is fix/OP-147054_add-select-option-at-binding-roomsensor.

   This removed the need to manually type IDs in the binding form. Users can select available sensors and rooms from fetched data, reducing input mistakes..

G. Settings, CORS, Docker, and Broker

1. **[OP-146049][API] Create Endpoint to check Server Status**

   This endpoint is useful for local development, Docker deployment, and operational checks because it confirms the server is running without calling business endpoints.

2. **[OP-146179][Docker] Create Dockerfile and DockerCompose for IotGateway**

   Containerized the IoT Gateway API and web app. The related commits added Dockerfiles, Docker Compose setup, Nginx config, .dockerignore files, SQLite bind mount handling, and .gitignore updates.

   This made the project easier to run consistently across environments. Developers can start the API, web app, database mount, and broker with Docker Compose.

3. **[OP-146197][Broker] Create Mosquitto message broker integration**

   Added Mosquitto broker support and MQTT subscriber integration. The branch history includes Mosquitto config, MQTT/WebSocket listeners, MQTT appsettings, hosted subscriber service, authentication handling, Docker Compose wiring, and README updates.

   This enabled Raspberry Pi or other IoT publishers to send sensor data through MQTT. The API subscribes to the configured topic and forwards valid payloads into the ingestion service.

4. **[OP-147029][Config] Adding Local URL to enable CORS**

   Added local development URLs to CORS configuration. The related history includes CORS allowed origins for localhost, loopback, local network IP, and HTTPS Meeting Room web origins.

   This allows the web dashboard and SignalR client to communicate with the API during local and Docker-based development.

H. SignalR and Realtime Monitoring

1. **[OP-146573][Hub] Create SensorHub for SignalR Controller**

   Implemented SensorHub for realtime sensor updates. The related branch added SignalR package support, hub registration, sensor group join/leave behavior, frontend SignalR dependency/service, CORS adjustments, and MQTT subscriber broadcasting.

   This allows the frontend to receive live sensor telemetry without polling. Each single-device page can join a sensor-specific SignalR group.

2. **[OP-147236][API/WEB] Handle Condition If Raspi Connection Lost**

   Implemented Raspberry Pi/sensor disconnection handling. The related commits added MqttSensorRequest, MQTT connection-state tracking, inactivity timeout handling, global using updates, CORS/config fixes, Docker environment updates, and live message display.

   This improves realtime monitoring because the dashboard can show inactive or disconnected sensor messages when devices stop publishing or explicitly report disconnection.

3. **[OP-147286][WEB] Fix Connected Component at IotGateway Panel**

   Fixed the connected/status UI on the IoT Gateway panel. The related branch fix/OP-147286_fix-connected-component removes unused disconnect/status display behavior and updates documentation for sensor status handling.

   This matches the latest single-device page behavior where the old connected badge is removed and the page keeps explicit controls such as disconnect/reconnect behavior for the SignalR connection.

I. Web Dashboard and User Experience

1. **[OP-147056][WEB] Implement GET api/user/{guid} for User Data**

   Implemented frontend/backend current user retrieval behavior. The branch name appears as OP-14756_get-user-guid-web, but OpenProject shows OP-147056.

   The implementation added user services/hooks and updated dashboard layout/page usage. The backend now retrieves the current user from JWT claims through /api/user/me.

2. **[OP-147085][WEB] Delete Unused Service and Separate Chart Usage**

   Cleaned unused frontend services and improved development integration behavior. Related commits remove unused sensor hook usage, update room-sensor delete endpoints, configure Meeting Room API HTTP handling, and update Docker environment variables.

   This reduced frontend service confusion and kept web/API contracts aligned. It also helped stabilize local development around Meeting Room API and room-sensor routes.

J. Testing

1. **[OP-146894][Test] Create Unit Test For IotGateway.Api**

   Added unit test coverage for the IoT Gateway API. The related test branch created IotGateway.Tests, a repository test base using in-memory SQLite, controller tests, service tests, repository tests, and solution updates.

   These tests improve confidence when refactoring or extending the API. They cover important flows around auth, ingestion, sensors, settings, processing, and repository behavior.

3. Rifa Fauziah

A. Sensor HLK-LD2401B Reader

1. **[OP-145052] Setup Repository IoT Sensor**

   This merge pushes the structure of the folder for IoT sensor application. The project was setup as a .NET application that can run on Raspberry Pi and communicate with LD2401B radar sensor through serial communication.

2. **[OP-145340] [Console] Implement Radar Sensor Integration**

   This feature implements the core integration between the console application and the HLK-LD2410B radar sensor. The application opens a serial connection to the sensor, reads incoming bytes, detects complete LD2410B data frames, and parses the frame content into meaningful occupancy information. The implementation supports engineering mode so that detailed sensor data can be read and gate sensitivity can be configured. This integration allows the Raspberry Pi application to continuously monitor whether a meeting room is occupied based on radar sensor output.

3. **[OP-145711] Fix IoT Sensor Payload**

   This task improves the payload structure sent from the IoT Sensor application to the IoT Gateway. The payload fields were adjusted to match the expected gateway contract, including JSON property names and data types. This ensures that the gateway can correctly deserialize the incoming request and process the sensor update without validation errors. The fix also keeps the application

focused on sending only the required telemetry data instead of exposing unnecessary internal sensor details.

4. **[OP-145748] Add Debug for Error Response**
   This task adds debugging support for API error responses when the IoT Sensor application sends telemetry to the gateway. The API sender now prints the HTTP status code, response body, and validation error details when a request fails. This makes it easier to identify whether a failure is caused by an invalid payload, missing token, gateway validation rule, or network timeout. The additional debug output is useful during integration testing because the sensor application runs continuously and needs clear feedback when a send operation fails.

5. **[OP-145957] Fix Occupancy Detection Check**
   This task improves the occupancy detection logic used when converting LD2410B target states into the IsOccupied payload value. This prevents invalid or non-occupancy sensor states from being treated as room occupancy. The fix makes the telemetry result more reliable and better aligned with the LD2410B protocol.

6. **[OP-145997] Add Config File**
   This feature adds configuration support to avoid hardcoding runtime values directly in the application flow. Configuration values such as serial port name, baud rate, gateway endpoint, token, and sensor-related settings can be separated from the main application logic. This makes the application easier to deploy on different machines, especially when moving from local development on macOS to execution on a Raspberry Pi. The configuration file also improves maintainability because environment-specific values can be changed without modifying the core program code.

7. **[OP-145998] Add Logger To The Program**
   This feature adds structured logging to the IoT Sensor application. Logging is used to record important runtime events such as serial connection startup, sensor configuration, parsed sensor readings, outgoing payloads, API responses, and runtime errors. This improves observability and makes the program easier to debug when running on Raspberry Pi. With logging in place, issues such as noisy serial data, failed API requests, delayed reads, or invalid payloads can be diagnosed more clearly.

8. **[OP-146064] Improve LD2410B Sensor Runtime Reliability**
   This task improves the reliability of the LD2410B runtime reading process. The implementation handles noisy serial frames, validates frame headers and tails,

checks frame length, and applies recovery logic for observed high-bit noise during testing. The parser also includes debug output for raw frames, recovered frames, target states, distance values, and energy values. These improvements help the application continue running during unstable serial communication while still making it possible to identify when sensor data is recovered or potentially unreliable.

9. **[OP-146208] Fix Delayed Occupancy State Caused by Sensor Read Loop Delay**
   This task fixes delayed occupancy updates caused by the application blocking the sensor read loop while waiting for HTTP requests or delays. The previous behavior allowed serial frames to accumulate in the buffer while the program waited for API calls to finish, causing the application to read stale sensor data. The fix separates sensor reading concerns from slow external operations and ensures that stale buffered data does not incorrectly represent the current room state. This improves the accuracy of occupancy changes, especially when a room changes from occupied to empty or from empty to occupied.

10. **[OP-146211] Implement MQTT Publisher for Sensor Updates**
    This feature adds MQTT publishing support for sensor updates. Instead of relying only on HTTP requests, the application can publish occupancy telemetry to an MQTT topic so that the IoT Gateway or other subscribers can receive updates in a lightweight and event-driven way. The MQTT publisher prepares the same sensor payload and sends it to the configured broker topic. This approach is suitable for IoT environments because MQTT is designed for continuous telemetry and can reduce the overhead caused by repeated HTTP requests.

11. **[OP-146895] Implement Unit Testing**
    This task introduces unit tests for the IoT Sensor application. The tests focus on critical logic such as frame parsing, byte conversion, occupancy state mapping, payload creation, and service behavior. By testing the parser and helper utilities separately from the physical sensor, the application can verify protocol handling without always requiring hardware access. These tests help prevent regressions when modifying serial parsing, recovery behavior, or payload formatting.

12. **[OP-146898] Fix Dependency Injection / Dependency Design**
    This task improves the dependency design of the IoT Sensor application by separating responsibilities between services and reducing tight coupling. Sensor reading, command sending, payload publishing, configuration, and logging are organized into dedicated services. This makes each component easier to understand, test, and maintain. The dependency design also prepares the

application for future changes, such as switching from HTTP to MQTT or replacing hardcoded runtime values with configuration-based values.

13. **[OP-146946] Configure DI Container for Sensor Services**

    This task configures the dependency injection container for the IoT Sensor services. The application registers services such as the frame parser, sensor reader, command service, API sender, MQTT publisher, configuration provider, and logger so they can be resolved consistently at runtime. Using dependency injection keeps the program entry point cleaner and makes the application structure closer to a production-ready service. It also improves testability because dependencies can be replaced with mocks or test implementations during unit testing.

14. **[OP-147234] Add Sensor Connection Status in Payload**

    This update improves the sensor payload by adding sensor connection information. The payload now includes isSensorConnected and sensorMessage, allowing the IoT Gateway to know whether the sensor is actively sending valid data or experiencing an issue. When a valid sensor frame is parsed, the sensor is marked as connected and a success message is included in the payload. If the serial connection fails, sensor configuration fails, or no valid frame is received within the timeout period, the payload is published with a disconnected status and an appropriate message. This update also includes the sensor connection status and message in runtime logs, making sensor issues easier to identify during monitoring and debugging.

15. **[OP-147011] Set Distances to Zero When No Target**

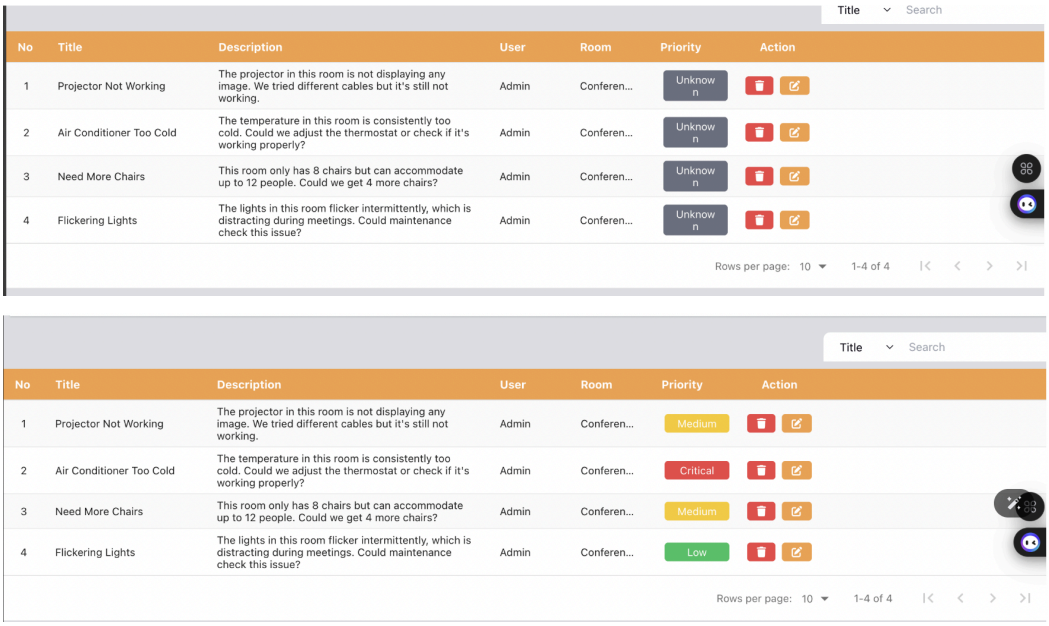
    This fix improves the accuracy of the sensor payload when the LD2410 radar reports no detected target. Previously, the sensor frame could still contain moving or stationary distance values even when targetState = 0x00. These values could be stale or noisy, but the application might still pass them into the payload. The update treats targetState = 0x00 as a valid no-target frame. When no target is detected, the application keeps publishing the payload with IsOccupied = false, but sets both MovingTargetDistance and StationaryTargetDistance to 0. Normal distance parsing is still preserved for occupied states.

B. Report Management System Enhancement

1. **[OP-146578] Fix Report Priority Labels as Unknown**

   This update fixes an issue where report priority values were displayed as “Unknown” in the report list and update modal. The problem occurred because

the frontend did not correctly align its priority mapping with the enum values returned by the backend API. The implementation adds proper priority value mapping for Low, Medium, High, and Critical, ensuring that report priorities are displayed accurately and consistently across the report management interface.

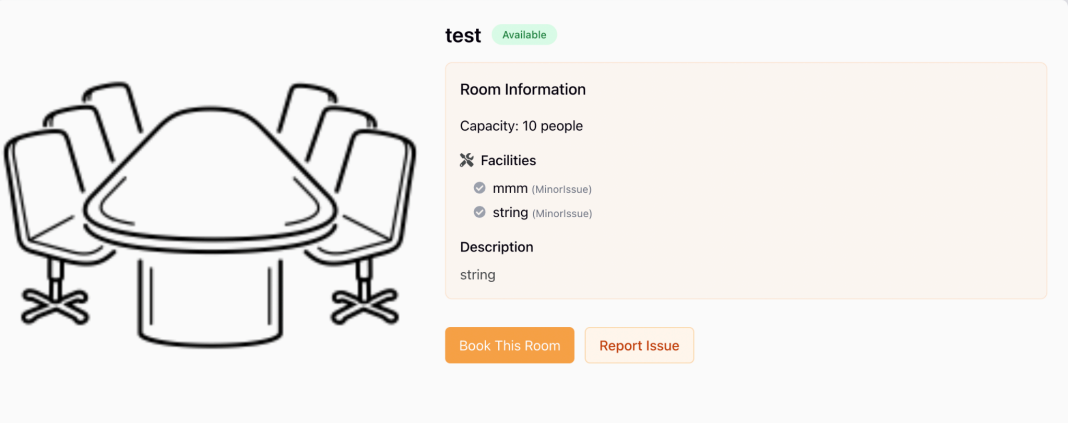


2. **[OP-147012] Add Admin Report Status Update UI**

   This feature adds an admin-facing interface for updating report statuses directly from the report list. Previously, report status data existed in the backend but was not clearly manageable from the frontend. With this update, administrators can update report statuses such as Pending, In Progress, Resolved, and Closed using a dropdown control in the report table. Regular users can only view the current status, ensuring that status management remains restricted to administrators.

3. **[OP-147021] Add Visible Report Issue Action on Room Detail**

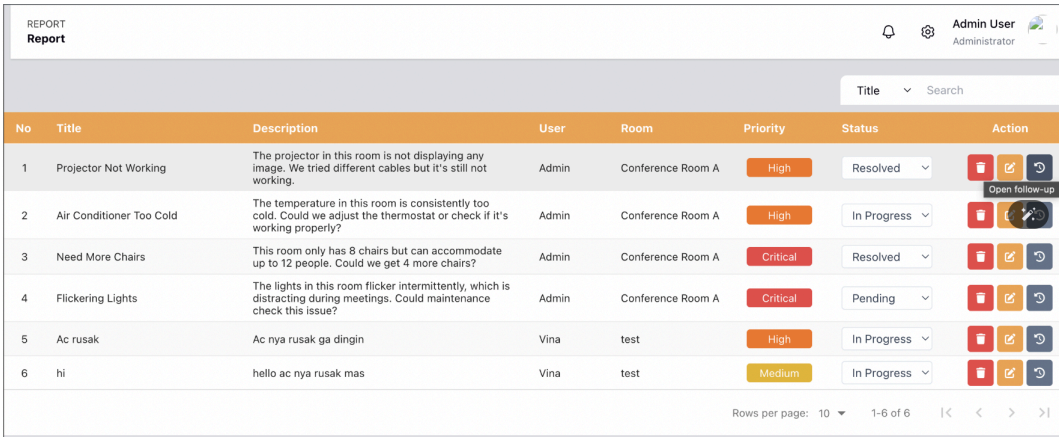
   This update improves the report submission experience by making the report issue action more visible on the Room Detail page. Previously, users could report room issues, but the action was not prominent enough and could be missed during normal room browsing. The implementation adds a clearer report issue button on the room detail view, allowing users to quickly submit room-related reports from the context of the room they are viewing.

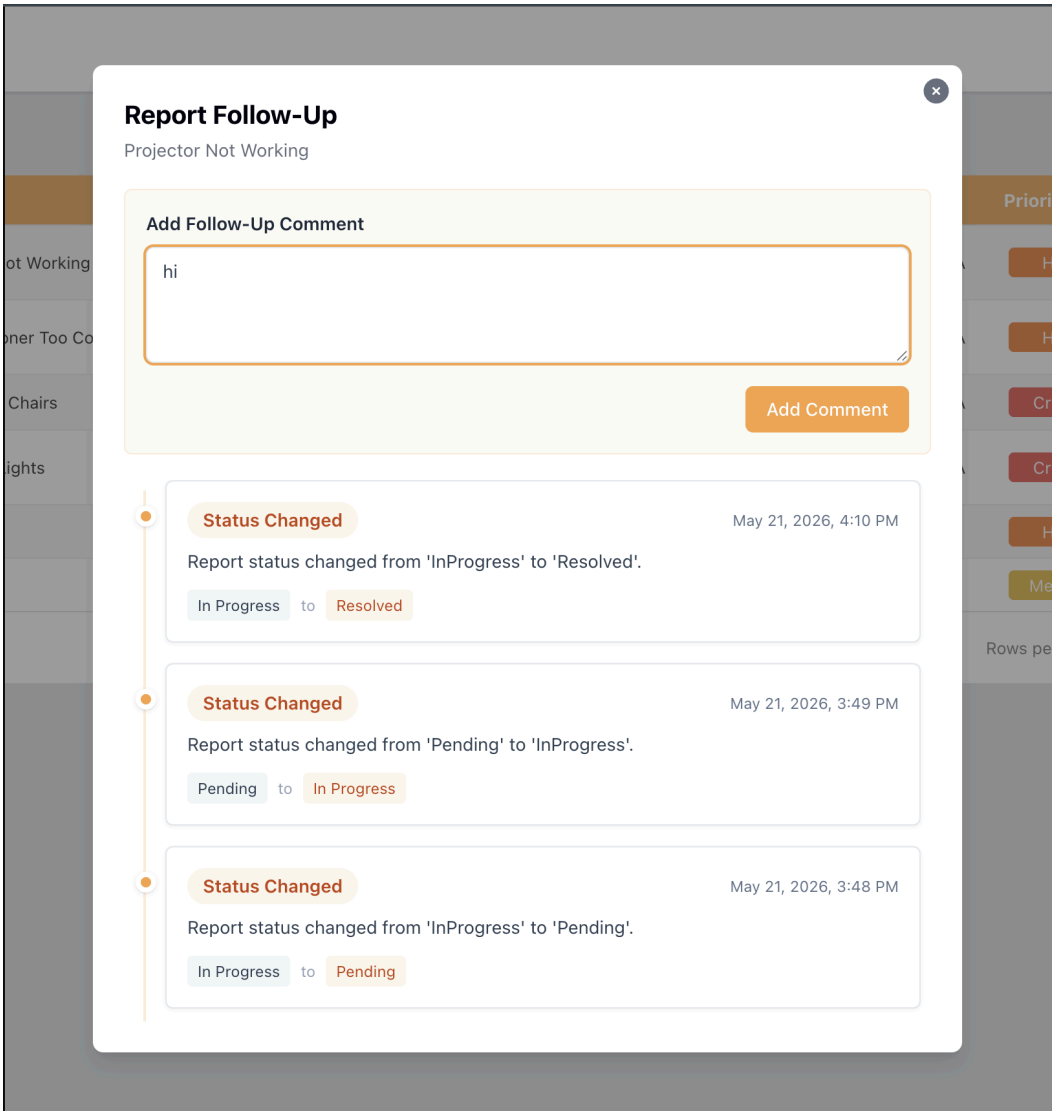


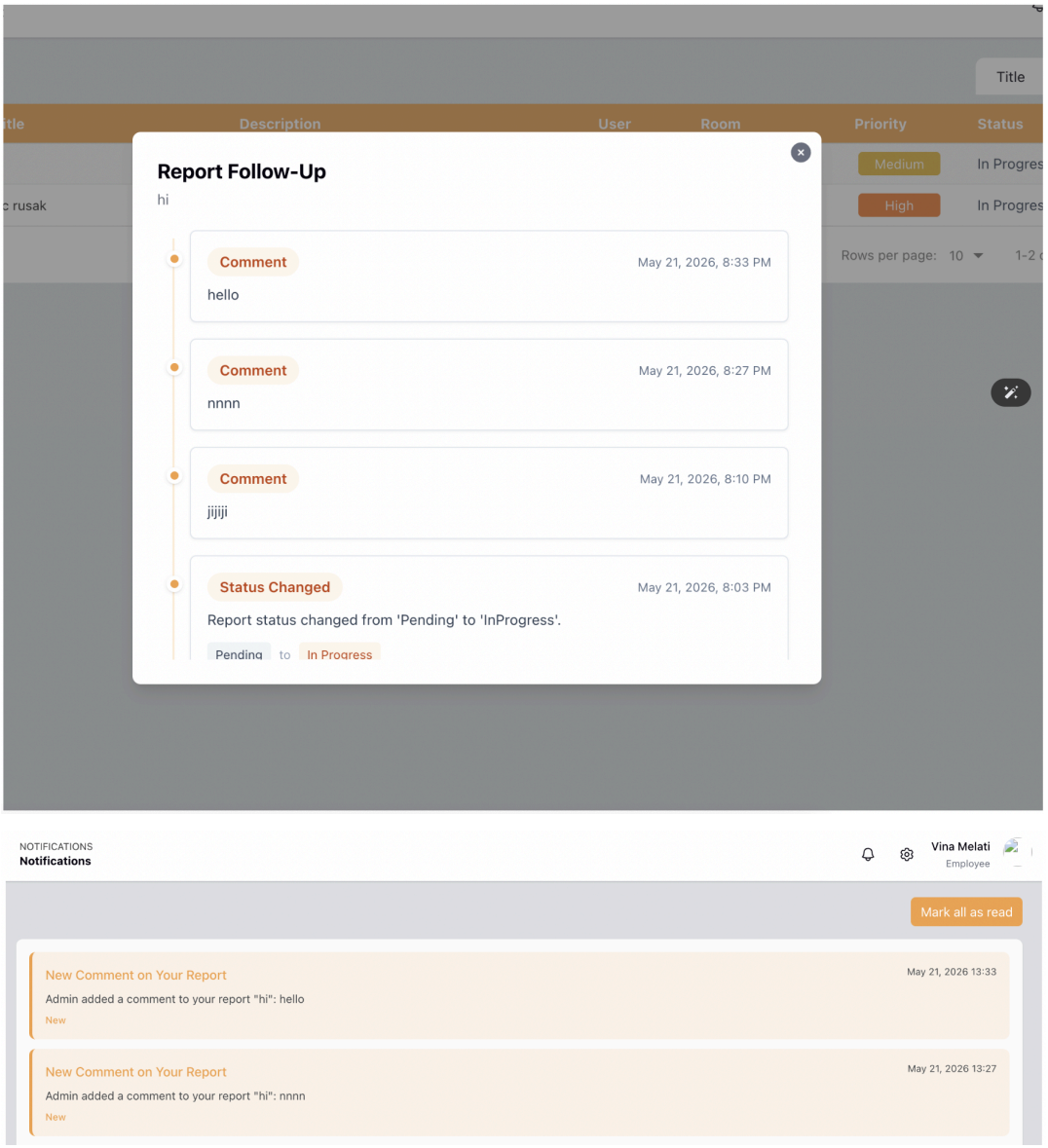
4. **[OP-147051] Add Report Follow-Up Comment Flow UI**

   This feature adds a report follow-up flow that allows administrators to communicate progress updates on submitted reports. A follow-up modal was added to the Reports page, displaying a chronological timeline of report events such as submission, status changes, detail updates, and admin comments. Administrators can add follow-up comments directly from the modal, while report owners receive an in-app notification when a new comment is added.

   The implementation integrates the frontend with the existing report follow-up APIs and enhances the backend notification flow for admin comments. This creates a clearer communication trail between administrators and report owners, improves issue tracking transparency, and makes the report resolution process easier to monitor.


REPORT
Report
Admin User
Administrator
Title
Search
No
Title
Description
User
Room
Priority
Status
Action
1
Projector Not Working
The projector in this room is not displaying any image. We tried different cables but it's still not working.
Admin
Conference Room A
High
Resolved
2
Air Conditioner Too Cold
The temperature in this room is consistently too cold. Could we adjust the thermostat or check if it's working properly?
Admin
Conference Room A
High
In Progress
3
Need More Chairs
This room only has 8 chairs but can accommodate up to 12 people. Could we get 4 more chairs?
Admin
Conference Room A
Critical
Resolved
4
Flickering Lights
The lights in this room flicker intermittently, which is distracting during meetings. Could maintenance check this issue?
Admin
Conference Room A
Critical
Pending
5
Ac rusak
Ac nya rusak ga dingin
Vina
test
High
In Progress
6
hi
hello ac nya rusak mas
Vina
test
Medium
In Progress
Open follow-up
Rows per page: 10
1-6 of 6


Report Follow-Up
Projector Not Working
Add Follow-Up Comment
hi
Add Comment
Status Changed
May 21, 2026, 4:10 PM
Report status changed from 'InProgress' to 'Resolved'.
In Progress
to
Resolved
Status Changed
May 21, 2026, 3:49 PM
Report status changed from 'Pending' to 'InProgress'.
Pending
to
In Progress
Status Changed
May 21, 2026, 3:48 PM
Report status changed from 'InProgress' to 'Pending'.
In Progress
to
Pending



5. **[OP-147239] Add Report Evidence Flow**

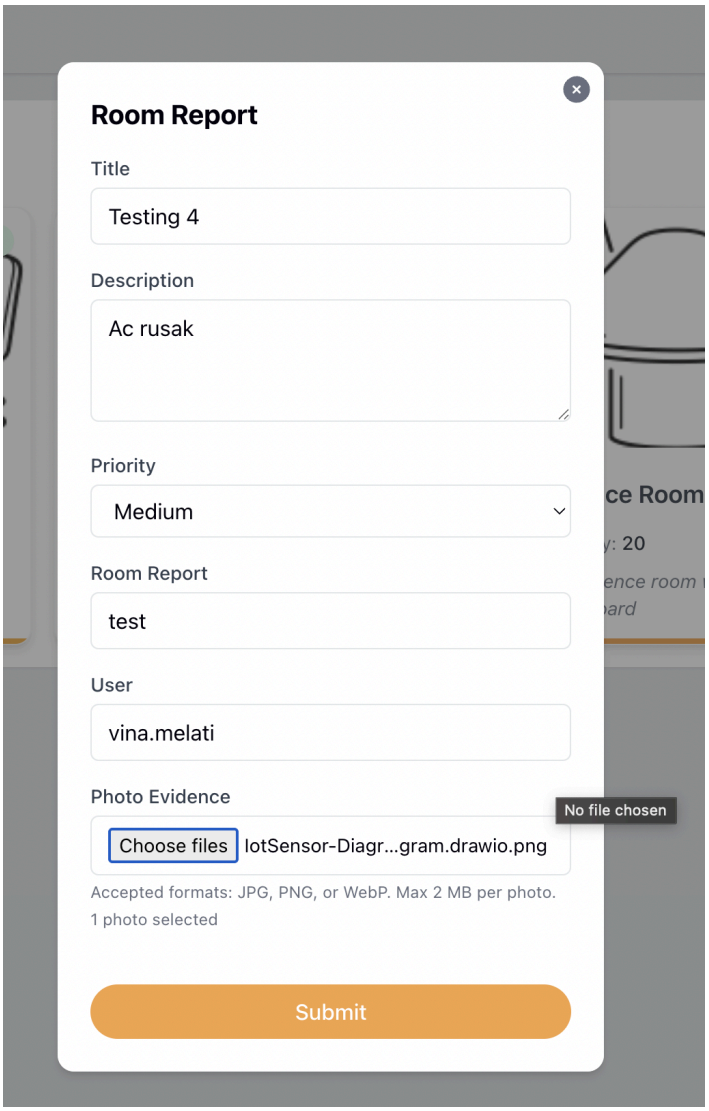
   This feature enhances the report management workflow by adding resolution feedback, user satisfaction survey, and photo evidence support. The update improves transparency between administrators and report owners by making report resolution details, user feedback, and supporting evidence easier to access and track.

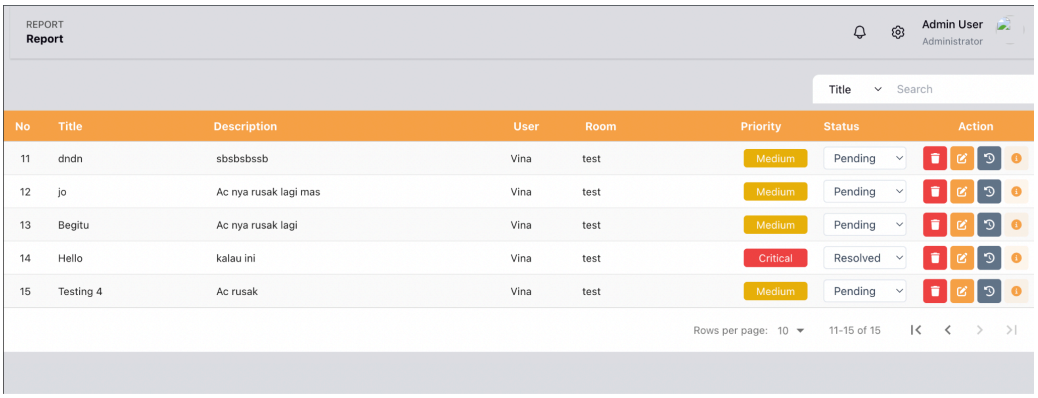
   Photo Evidence :

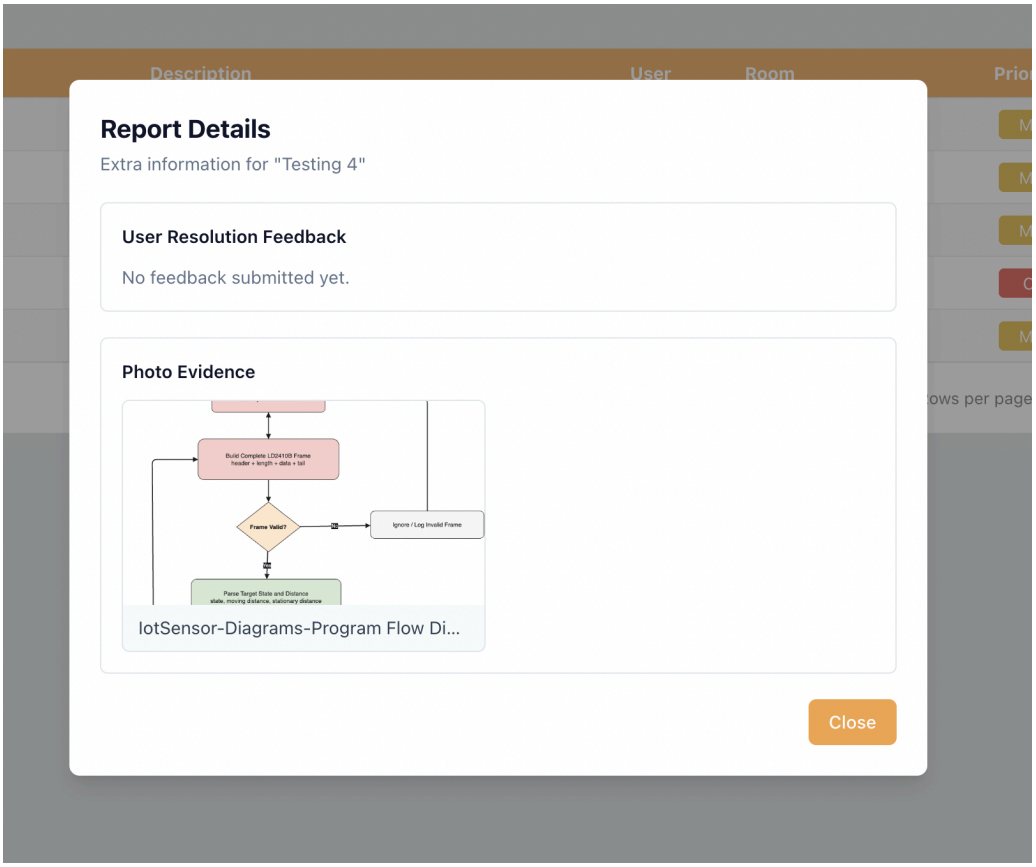
   This update adds optional photo evidence support when users submit room reports. Users can attach image evidence, such as photos of damaged facilities or broken equipment, while creating a report. The uploaded photos are linked to the report and can be viewed from the report detail modal.

   Photo evidence is not displayed directly in the main report table to keep the list layout clean and readable. Instead, it remains accessible from the report

details, making the supporting evidence available without cluttering the report overview.



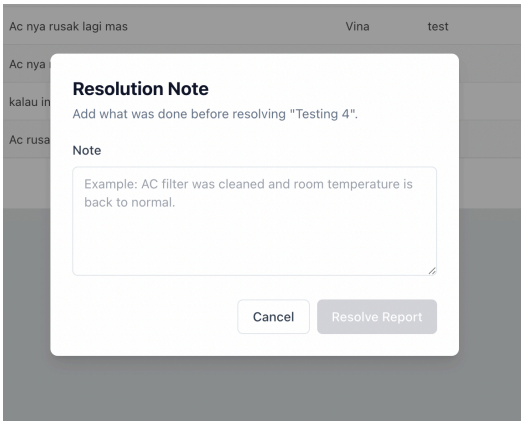


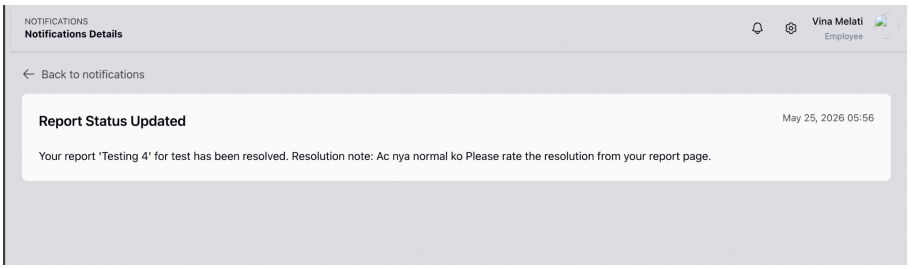


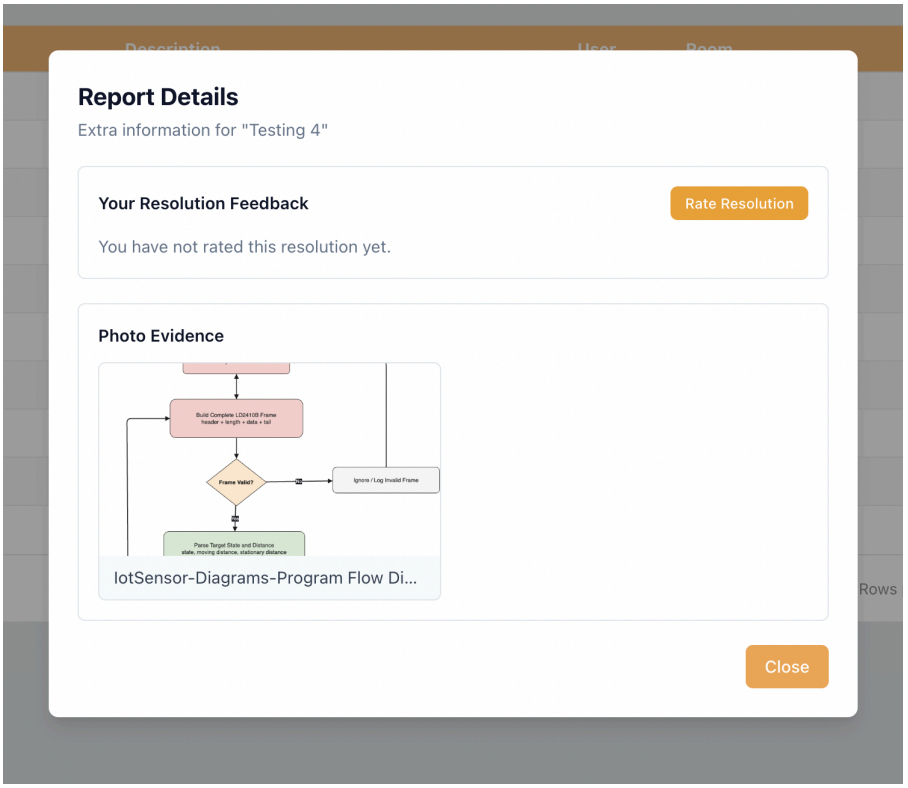
Resolution Feedback:

This update adds a resolution note flow when an administrator resolves a report. Previously, users could only see that a report status had changed to Resolved, but they did not receive clear information about what action was taken. With this enhancement, administrators must provide a resolution note when changing a report status to Resolved.

The resolution note is saved into the report follow-up timeline, allowing the report owner to review the action taken by the administrator. This improves accountability and gives users better context about how their report was handled.

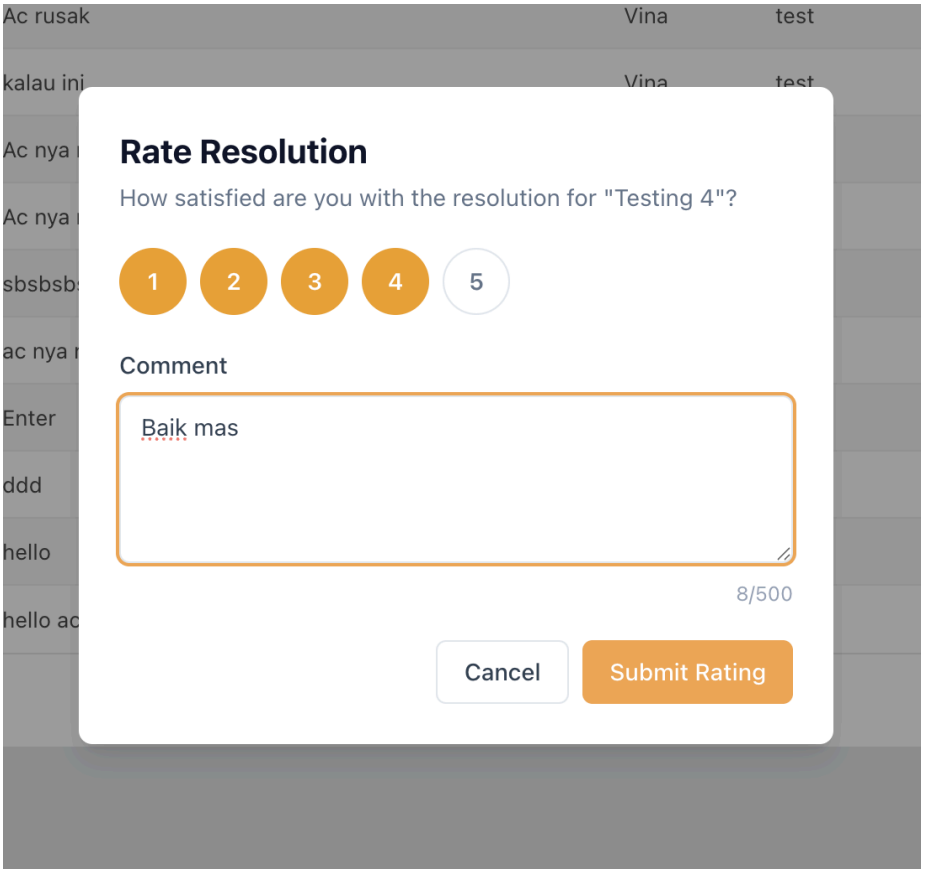


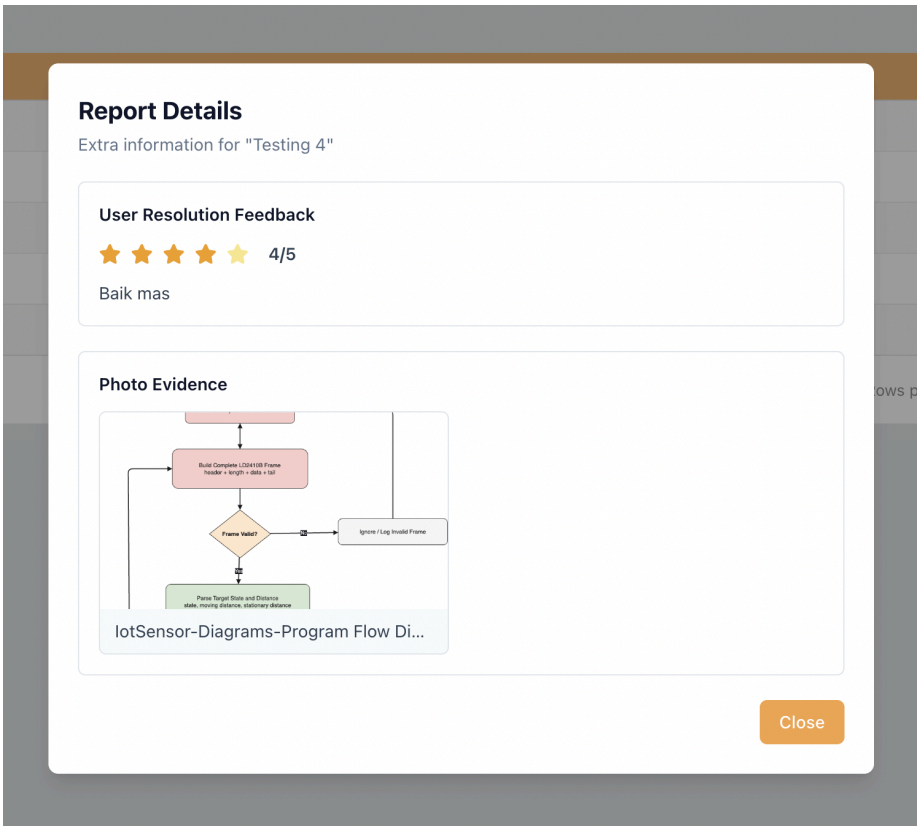




Satisfaction Survey :

This feature adds a satisfaction survey flow after a report has been resolved. Report owners can submit a one-time rating and optional comment to evaluate the resolution. The feedback is stored in a dedicated ReportSatisfaction table and displayed in the report detail modal. Administrators can view the submitted feedback, and an in-app notification is created when a user submits satisfaction feedback. This helps administrators monitor the quality of report handling and provides a clearer feedback loop between users and admins.


Ac rusak
Vina
test
kalau ini
Vina
test
Ac nya
Ac nya
sbsbsbs
ac nya
Enter
ddd
hello
hello ac
Rate Resolution
How satisfied are you with the resolution for "Testing 4"?
1
2
3
4
5
Comment
Baik mas
8/500
Cancel
Submit Rating


Report Details
Extra information for "Testing 4"
User Resolution Feedback
4/5
Baik mas
Photo Evidence
IotSensor-Diagrams-Program Flow Di...
Close

C. System and API Reliability Enhancement

1. **[OP-145676] Return ResultType as Readable Enum String in API Responses**

   This update improves API response readability by returning ResultType values as readable enum strings in API responses. Previously, result types could be difficult to interpret clearly during debugging and frontend integration. By exposing readable enum string values, developers can understand service response states more easily and handle API results with better clarity across the application.

2. **[OP-146438] Add Validation for DeviceHistory to Prevent Duplicate Device and Sensor Records**

   This update adds validation to the IoT device history flow to prevent duplicate records from being written when the same room and sensor/device combination already exists. Previously, repeated incoming data could create redundant history entries for the same relationship. The new validation ensures that duplicate records are skipped, improving data consistency and keeping device history records cleaner.

3. **[OP-147206] Update Existing IoT Device History Instead of Creating Duplicate Record**

   This update optimizes the IoT device history flow by preventing redundant records when a device sends continuous data with the same occupation status. The system now checks the latest recorded status for the given room and device combination and only update the 'IsOccupied' column if the same device

V. Future Development

[API]

- Use occupancy data to support no-show detection for booked rooms.
- Improve room availability status by combining reservation data and isOccupied device history.
- Add room utilization analytics endpoint based on device history data.
- Improve IoT API security with stronger device authentication, such as signed payloads or certificate-based identity.

[WEB]

- Display real-time or latest room occupancy status on the Meeting Room Booking room page.

- Add visual indicators for sensor connection status, such as connected, disconnected, or unknown.
- Add room utilization dashboard for administrators.
- Improve flow UI
- Improve IoT Gateway dashboard with clearer sensor health, room binding, and payload status.

[TEST]

- Increase unit test coverage for IoT-related services, controllers, and repositories.
- Add integration tests for invalid sensor payloads, failed authentication, and disconnected sensor status.
- Add regression tests for report flow enhancement and existing bug fixes.
- Add performance tests for multiple sensors publishing data at the same time.
- Add tests for occupancy-based future features such as no-show detection and auto-release.